VLSI Design: Course Project
Design and Simulation of a 5-bit Brent-Kung Adder

Navanshu Gupta
2024102072

1 Introduction

Addition is a fundamental operation in digital systems, significantly impacting the performance of arithmetic and logic units (ALUs), processors, and other digital circuits . Traditional ripple carry adders, while simple, suffer from high propagation delay due to the sequential dependency of carry generation. The Carry Look-Ahead Adder (CLA) addresses this limitation by employing a parallel approach to carry generation and propagation, resulting in reduced delay and enhanced performance for high-speed applications.

5 Bit Carry Look Ahead (CLA) Adder adds two 5 bit numbers and gives the final 5 bit sum along with the 6th bit carry. However unlike normal 5 bit Ripple Carry Adder which calculates each carry one at a time, CLA Adder calculates all the carry at the same time hence reducing the propagation delay.

2 Problem Statement

Design a 5-bit Carry Look Ahead (CLA) adder as shown in Fig. 1(i). Different modules of the CLA-adder are shown in Fig. 1(ii). Each output sum bit needs to drive an inverter of size $W_p/W_n = 20\lambda/10\lambda$, where $\lambda = 0.09\mu m$. As shown in Fig. 1(iii), consider that input bits are available before the rising edge of the clock and the output should be computed and present at the next rising edge of the clock.

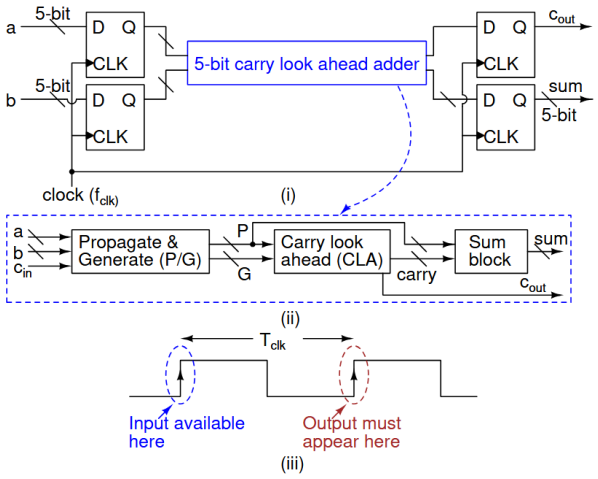


Figure 1: Problem Statement

3 Functioning of a Carry Look Ahead Adder

We define two terms, the generate (g_i) and the propogate (p_i) where

$$p_i = a_i \oplus b_i$$

$$g_i = a_i \cdot b_i$$

Then we calculate all carries in CLA block by following logic,

$$c_{i+1} = p_i \cdot c_i + g_i$$
$$S_i = p_i \oplus c_{i-1}$$

The computation of carry before the sum of that bit is reached significantly reduces the delay as the number of bits increases.

4 Proposed Architecture

We use a family of CLAs called Parallel Prefix Adders (PPAs). The two differ in the way their carry generation block is implemented.

The operation of Parallel Prefix Adders (PPAs) is based on reformulating binary addition as a prefix sum problem using a specialized operator.

- **The Prefix Operator ($\circ$):** A custom operator is defined to combine "generate" (g) and "propagate" (p) pairs. For two stages x and y, the operation is defined as:

  $$(g_x, p_x) \circ (g_y, p_y) = (g_x + p_x \cdot g_y, \quad p_x \cdot p_y)$$

  where $+$ represents logical OR and $\cdot$ represents logical AND.

- **Carry Generation:** The initial bitwise signals are $g_i = a_i \cdot b_i$ and $p_i = a_i \oplus b_i$. The cumulative group generate term G_i, derived from the recurrence $(G_i, P_i) = (g_i, p_i) \circ (G_{i-1}, P_{i-1})$, corresponds directly to the carry bit c_i for that stage.

While numerous Parallel Prefix Adder (PPA) architectures exist, three architectures are predominantly used due to their distinct optimization targets: Brent-Kung, Kogge-Stone, and Ladner-Fischer. Each represents a unique compromise between area, delay (logic depth), and wiring complexity (fan-out).

- **Brent-Kung Adder:** Minimizes the number of generate/propogate blocks and thus has the least area. But it has the largest delay compared to the other too.

- **Kogge-Stone Adder:** Widely considered the fastest PPA implementation, the Kogge-Stone adder achieves the theoretical minimum logic depth ($\log_2 n$) by computing carries in parallel This speed comes at a significant cost in silicon area and power.

- **Ladner-Fischer Adder:** The Ladner-Fischer architecture serves as a hybrid approach. Like the Kogge-Stone, it achieves the minimum logical depth ($\log_2 n$) for high speed. However, it modifies the internal prefix graph to reduce the overall number of nodes compared to Kogge-Stone. The primary drawback is high fan-out at certain logic levels.

4.1 The Brent-Kung Carry Lookahead Adder

The Brent-Kung adder is a parallel prefix adder designed primarily to minimize chip area and wiring complexity. This approach yields the minimum number of computation nodes and wiring tracks, significantly reducing the layout area. However, this efficiency comes at the cost of speed; the critical path delay is $2\log_2 n - 2$ logic levels, which is nearly double the $\log_2 n$ depth of the fastest parallel prefix adders. Given below is the carry tree and the circuit diagram for N=5 bits.

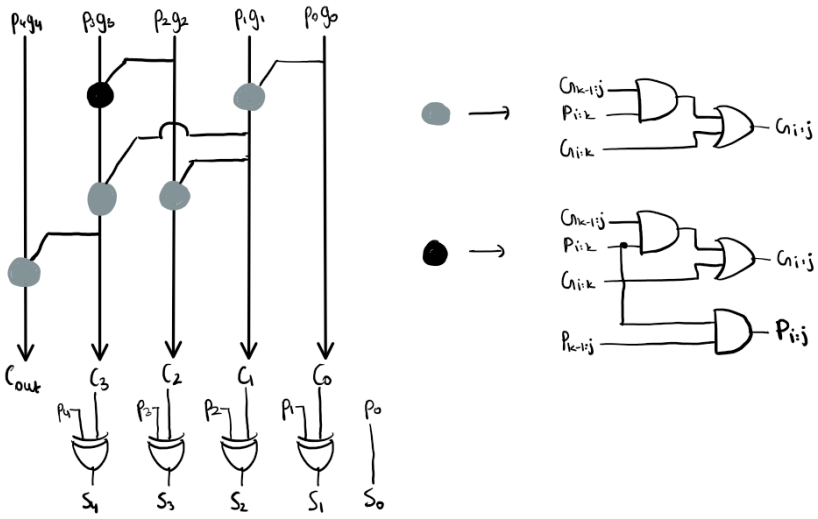


Figure 2: Implementation of Brent-Kung adder

Gates Used

Gates are sized accordingly such that the logical effort matches the reference minimum sized inverter.

- **Inverter** We use the static CMOS Implementation of the Inverter with $W_p = 5\lambda$ and $W_n = 10\lambda$

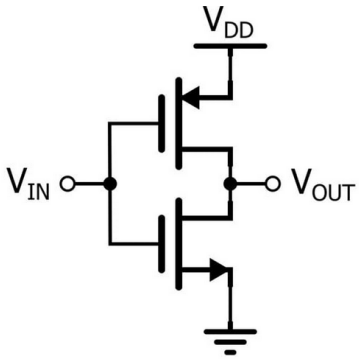


Figure 3: CMOS Implementation of the Inverter

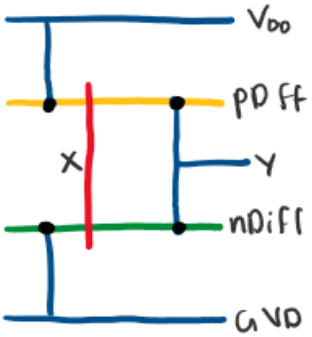


Figure 4: Stick Diagram of Inverter

- **NOR Gate** We use the static CMOS implementation of a 2-input NOR gate

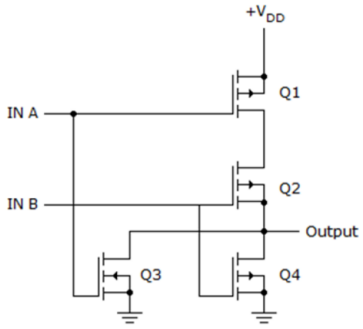


Figure 5: CMOS Implementation of the NOR Gate

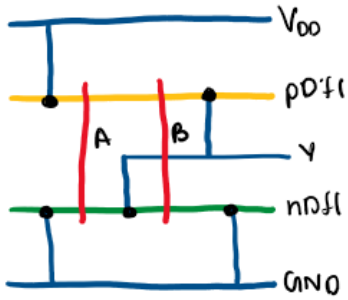


Figure 6: Stick Diagram of NOR gate

- **NAND Gate** We use the static CMOS implementation of a 2-input NAND gate

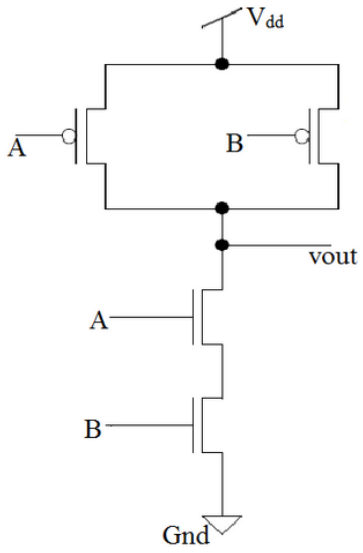


Figure 7: CMOS Implementation of the NAND Gate

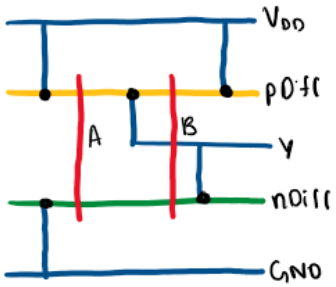


Figure 8: Stick Diagram of NAND Gate

XOR Gate can be Implemented by 4 NAND Gates using the following diagram.

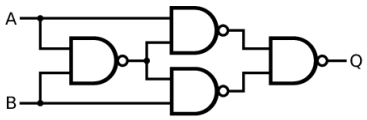


Figure 9: XOR Implementation using NAND

4.2 D Flip flop

We are using a True Phase Single Clock(TSPC) positive edge triggered flipflop which has the following circuit diagram:

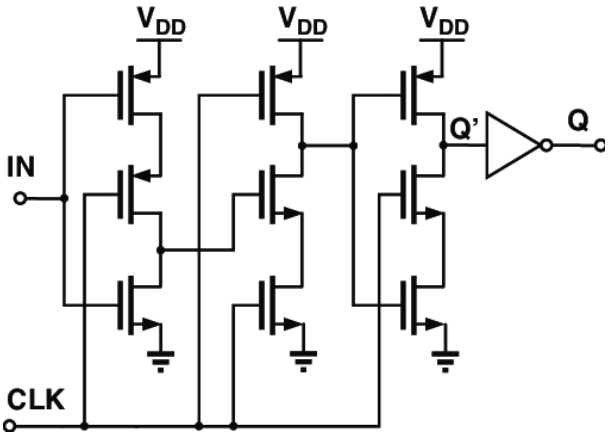


Figure 10: TSPC-FF Circuit Diagram

The True Single Phase Clock (TSPC) flip-flop is favored in high-speed designs because it eliminates the need for complementary clock signals, effectively removing skew problems associated with clock inversion. This simplification reduces the clock distribution network's complexity and transistor count, enabling higher operating frequencies and a more compact layout.

5 Pre-layout simulations

5.1 Adder simulation

We first run `.op` simulations for certain values of $A = \{a_4a_3a_2a_1a_0\}$ and $B = \{b_4b_3b_2b_1b_0\}$.

- Say A=10010 and B=11010. Then:

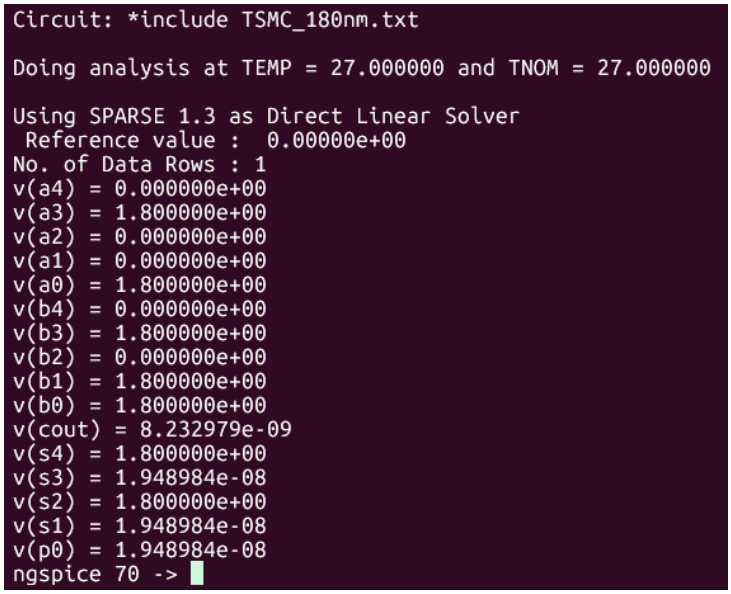


Figure 11: .op output for a particular combination of numbers

It is observed that the sum is exactly as expected, S=10100 and $C_{out} = 0$

- Say B=11111 and A=00001

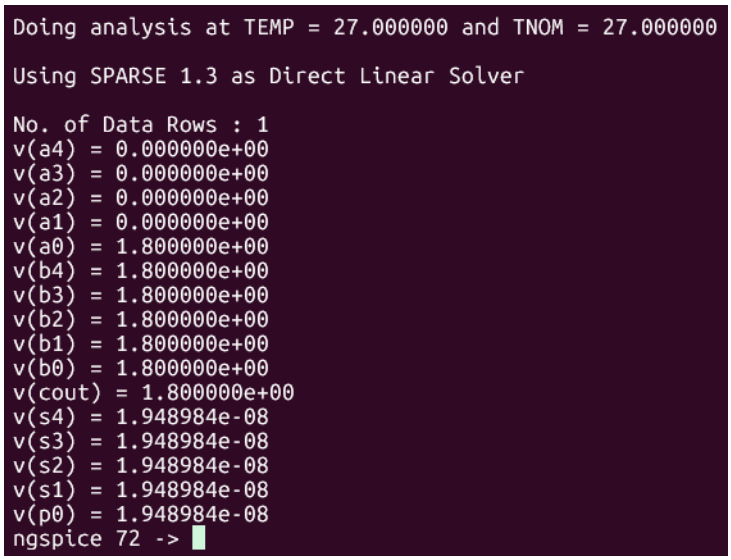


Figure 12: .op output for a particular combination of numbers

Delay analysis

Lets classify the worst and best possible delay case for the adder blocks t_{pdmax} and t_{pdmin}. The worst case for the delay is when the carry travels from the first bit to C_{out}, that is when A=00000 and B=11111 changes to A=00001 and B=11111. This instance will cause the largest delay for the sum to appear. We can measure the delay using the following lines of code

```
1     meas tran t_in when v(a0)=0.9 RISE=1
2     meas tran t_out when v(S4)=0.9 FALL=1
```

```
 Reference value :  4.95793e-07
No. of Data Rows : 5455
t_in                =  5.000000e-11
t_out               =  8.950509e-10
delay = 8.450509e-10
```

Figure 13: Measured delay for the adder block

Thus, $t_{pdmax} = 0.845ns$

Similarly, The best case delay is when A=00000 and B=00000 changes to A=00001 and B=00000. This instance will cause the smallest delay for the sum to appear since there is no carries generated.

```
 Reference value :  4.99856e-07
No. of Data Rows : 5405
t_in                =  5.000000e-11
t_out               =  1.386401e-10
delay = 8.864010e-11
```

Figure 14: Measured delay for the adder block

Thus, $t_{pdmin} = 0.086ns$

5.2 Flip-Flop simulation

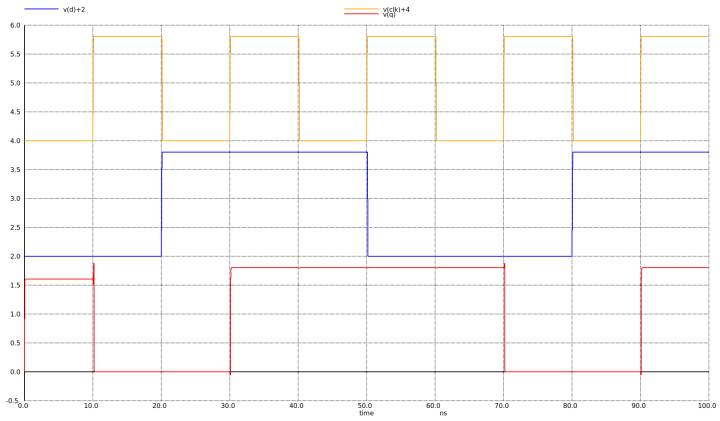


Figure 15: Working of a TSPC positive edge triggered flipflop

Clearly, the data changes at the positive edge of the clock.

Delay, setup and hold time analysis

The parameter t_{pcq} (Propagation Clock-to-Q) is defined as the time delay from the active clock edge until the output signal Q is guaranteed to be stable and valid. From our measurements, we get:

```
No. of Data Rows : 10071
tpcq_hl             =  1.341823e-10 targ=  7.018418e-08 trig=  7.005000e-08
tpcq_lh             =  7.475406e-11 targ=  3.012475e-08 trig=  3.005000e-08
```

Figure 16: Delay values for TSPC FlipFlop

Thus, $t_{pcqmax} = 0.134ns$ and $t_{pcqmin} = 0.075ns$

Setup time is the minimum amount of time that the data input (D) must be stable and valid before the arrival of the clock edge to ensure proper operation of the flip-flop. If the data changes within this time window, it may lead to incorrect behavior or undefined states.

By changing the shift of the pulse D and then measuring the exact difference when Q doesn't respond to D gives us the setup time. Thus, $t_{setup} = 0.172ns$

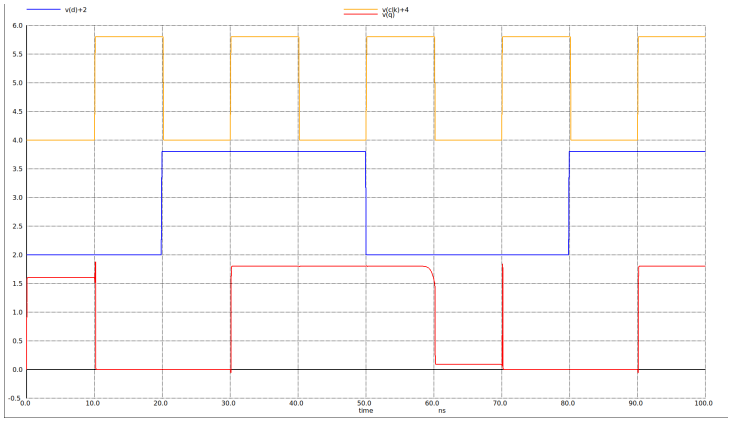


Figure 17: The exact shift in the signal D such that Q gets corrupted.

Similarly, $t_{hold} = 0.121ns$

5.3 Final Circuit Simulation

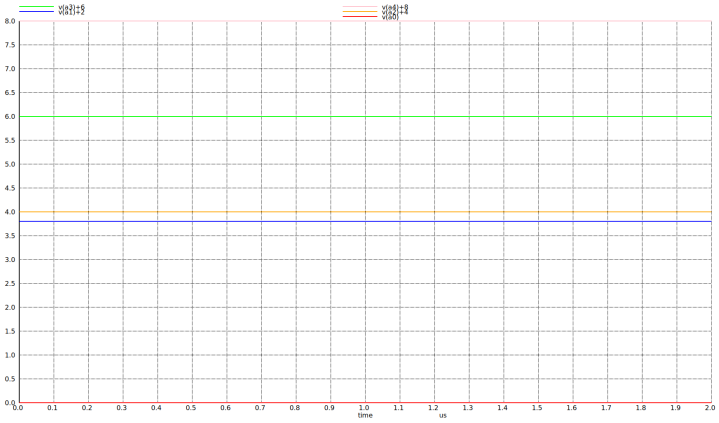


Figure 18: Waveforms of A=00010

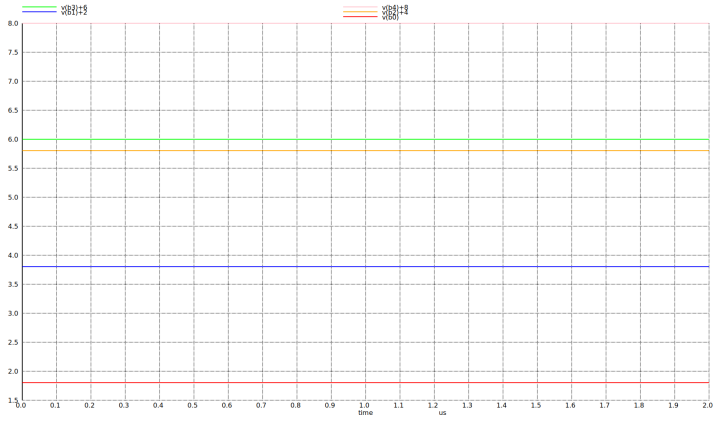


Figure 19: Waveforms of B=00111

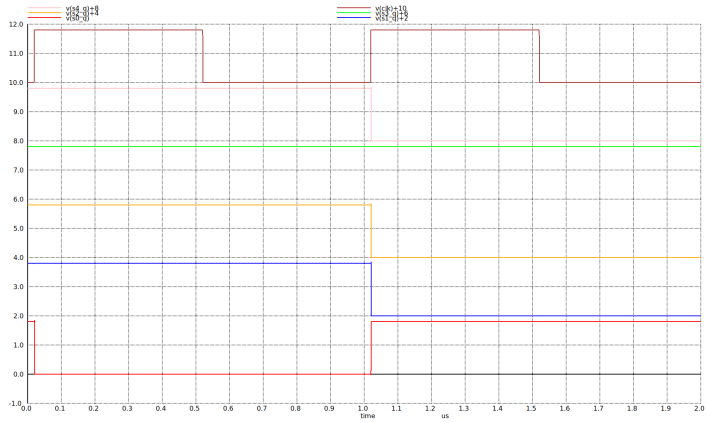


Figure 20: Output waveforms

Clearly evident that the values of S changes at the next positive edge of the clock from where the information was available.

5.4 T_{clk} analysis

We know that,

$$T_{clk} \geq t_{pd,max} + t_{pcq,max} + t_{su} \tag{1}$$

$$t_h \leq t_{pd,min} + t_{pcq,min} \tag{2}$$

Thus, $T_{clk} \geq 0.845 + 0.134 + 0.172$ and $T_{max} = 1.151ns$ so $F_{max} = 868.8Mhz$
Checking Hold time violation, $t_h > 0.086 + 0.075$. There, it does not violate this condition.

6 Layout

All the blocks shown here have been individually tested and they give the proper functional output that they were intended to. These are the layouts of the different blocks in magic.

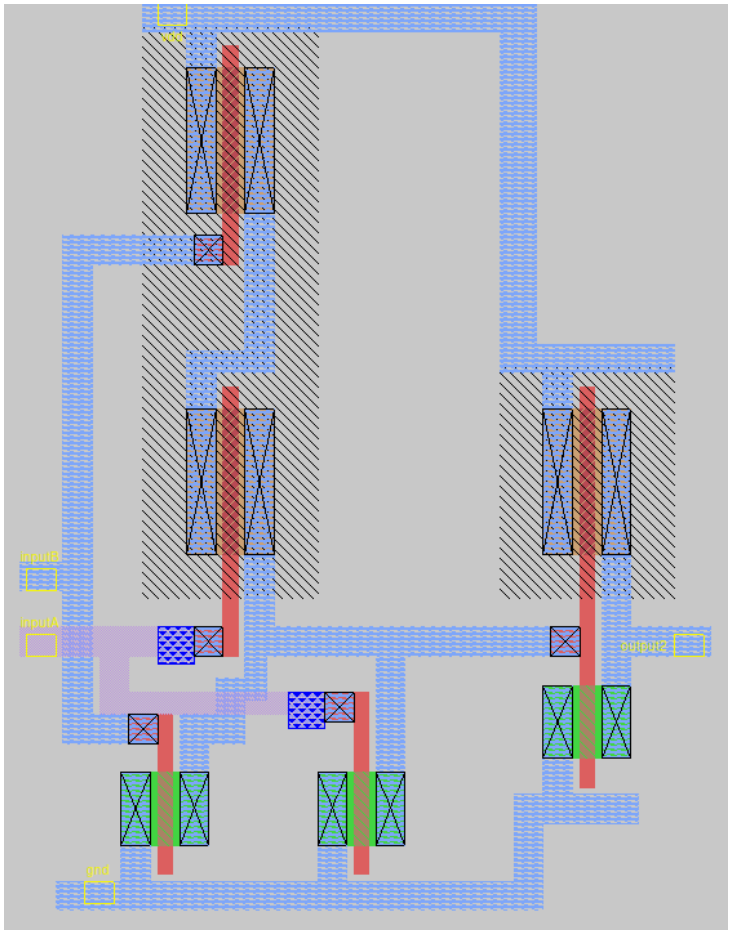


Figure 21: CMOS OR Gate Layout

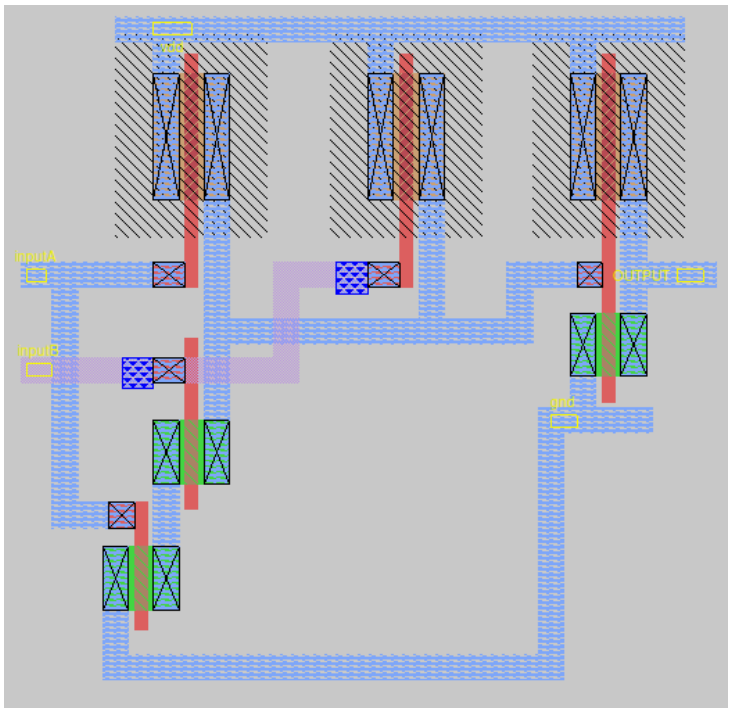


Figure 22: CMOS AND Gate Layout

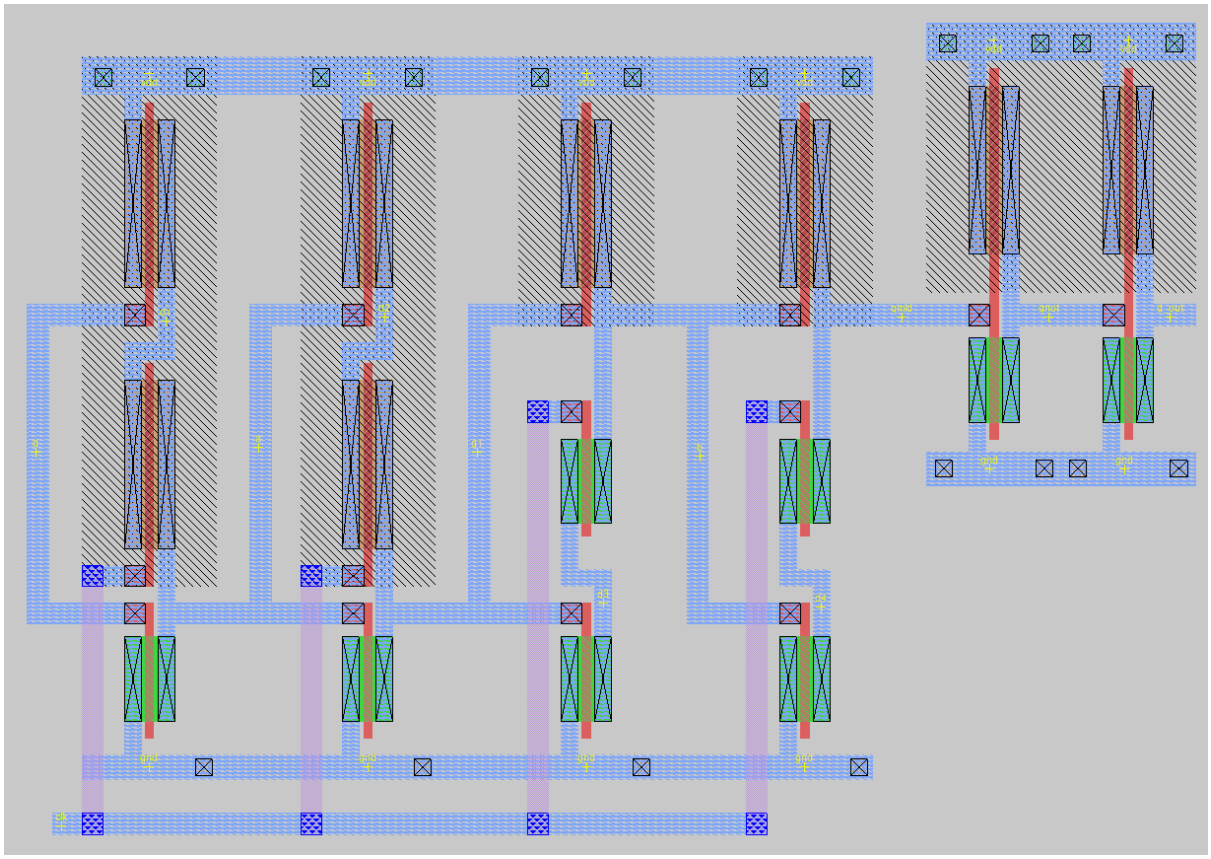


Figure 23: TSPC DFF Layout

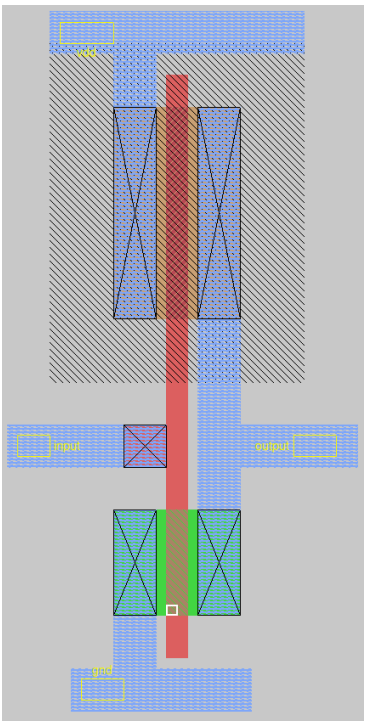


Figure 24: CMOS Inverter Layout

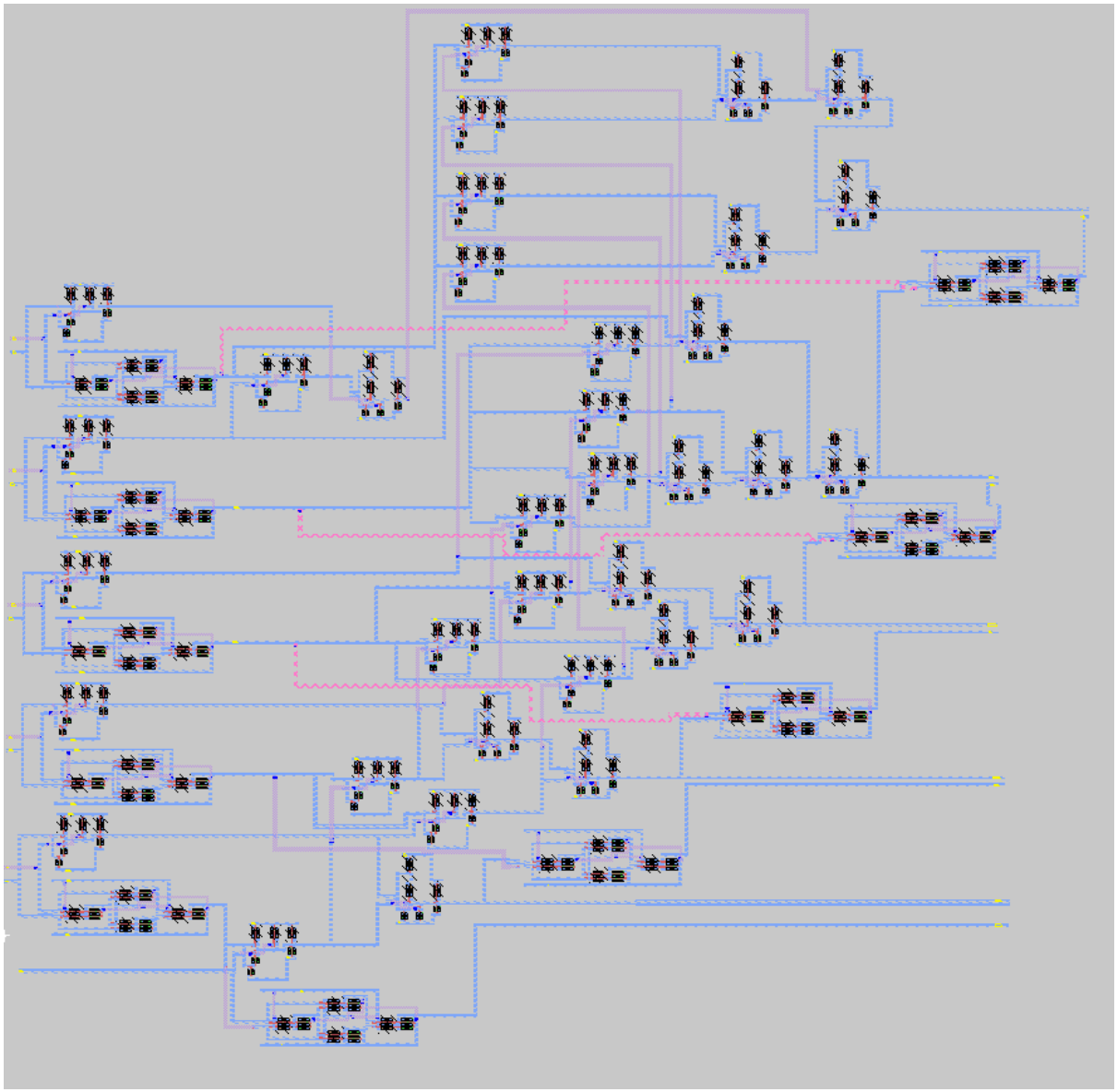


Figure 25: Adder block layout in magic

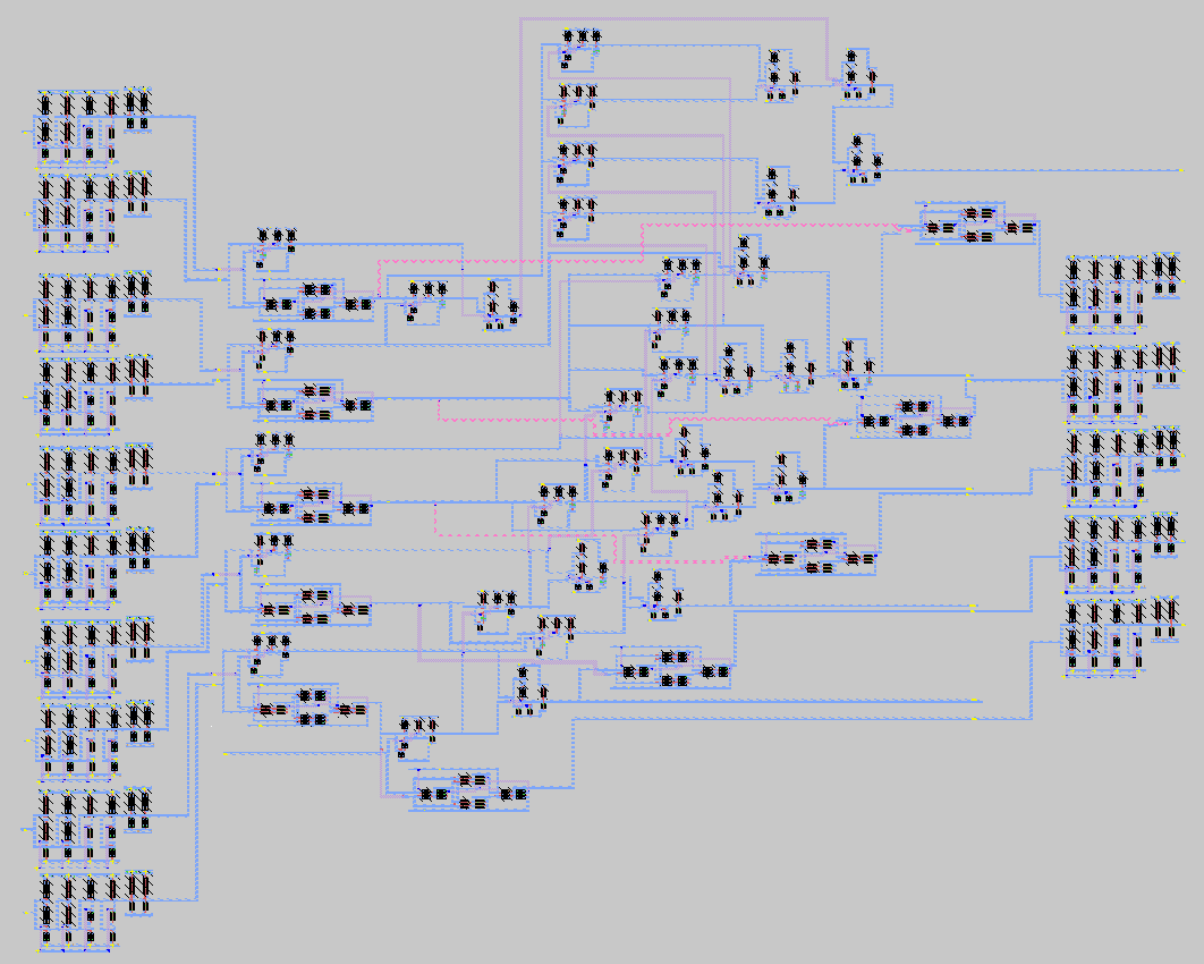


Figure 26: Entire block layout in magic

7 Post-Layout Simulations

7.1 Delay, setup and hold time analysis

```
No. of Data Rows : 10071
tpcq_hl              =  1.508572e-10 targ=  7.020086e-08 trig=  7.005000e-08
tpcq_lh              =  9.195413e-11 targ=  3.014195e-08 trig=  3.005000e-08
ngspice 49 ->
```

Figure 27: Post-layout simulations for flip-flop

The delay, setup and hold time are calculated similar to the pre-layout versions. $t_{pcq,min} = 0.092ns$
$t_{pcq,max} = 0.150ns$
$t_{setup} = 0.230ns$
$t_{hold} = 0.182ns$

Similarly, the new delays for the adder blocks are: $t_{pd,max} = 0.525ns$
$t_{pd,min} = 0.150ns$

Thus, $T_{clk} \geq 0.525 + 0.150 + 0.230$ and $T_{max} = 0.905ns$ so $F_{max} = 1.1Ghz$
This also does not violate hold time violation.

8 Verilog Analysis

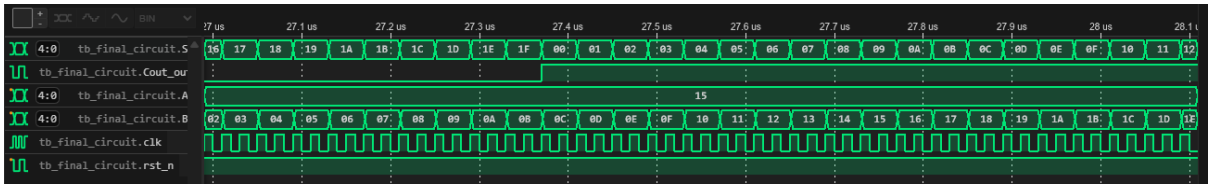


Figure 28: GTK-wave outputs for the circuit

9 FPGA Implementation

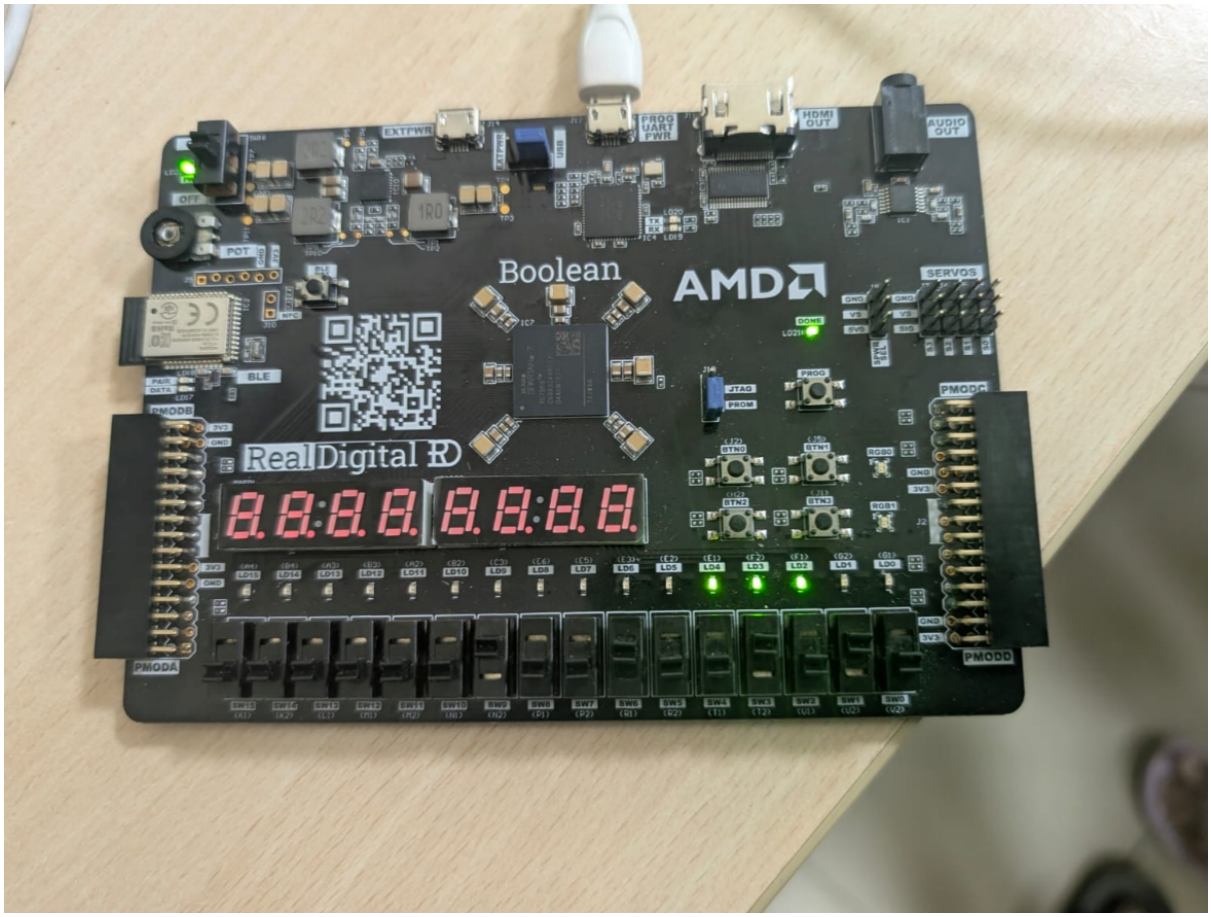


Figure 29: FPGA photo